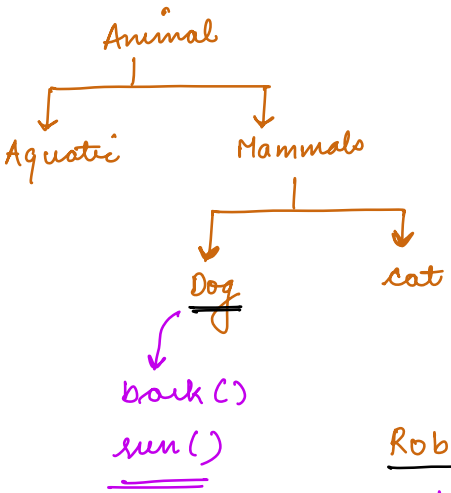
Interfaces & Abstract classes

Interfaces



categorizes on the basis of logical / physical relationship.

Robot Dog
bark()
run()

Organise a Race ⟶ which can run

List < ? > participants

Animal ? ✗
Dog ? ✗
Robot Dog ? ✗
Object

categorizes on the basis of behaviours.

↓

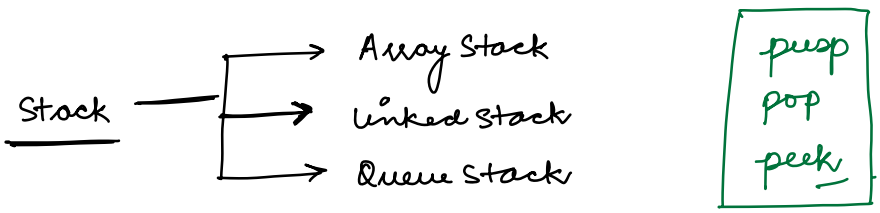
Interface

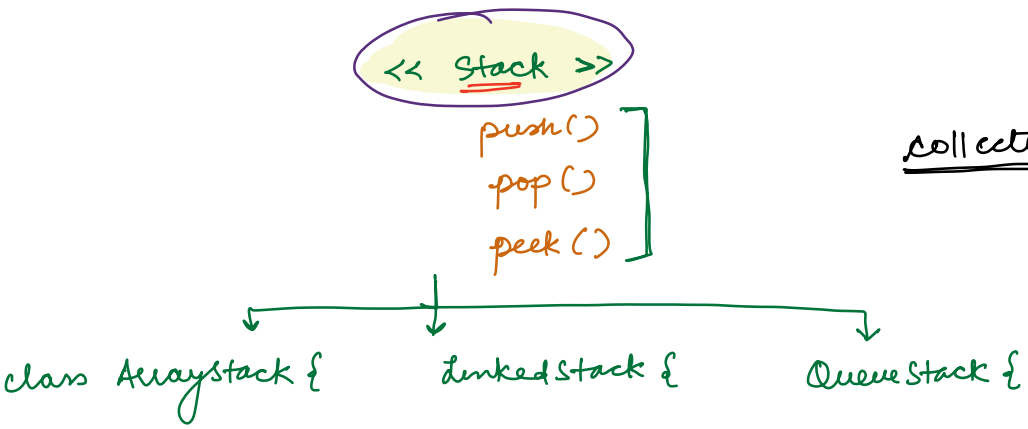
```
interface Runner {
    public abstract void run();
}

class RoboticDog implements Runner {

    void run() {
        ==
    }

}
```

abstract

contract → run()

class implementing an interface have to define all the methods declared in interface



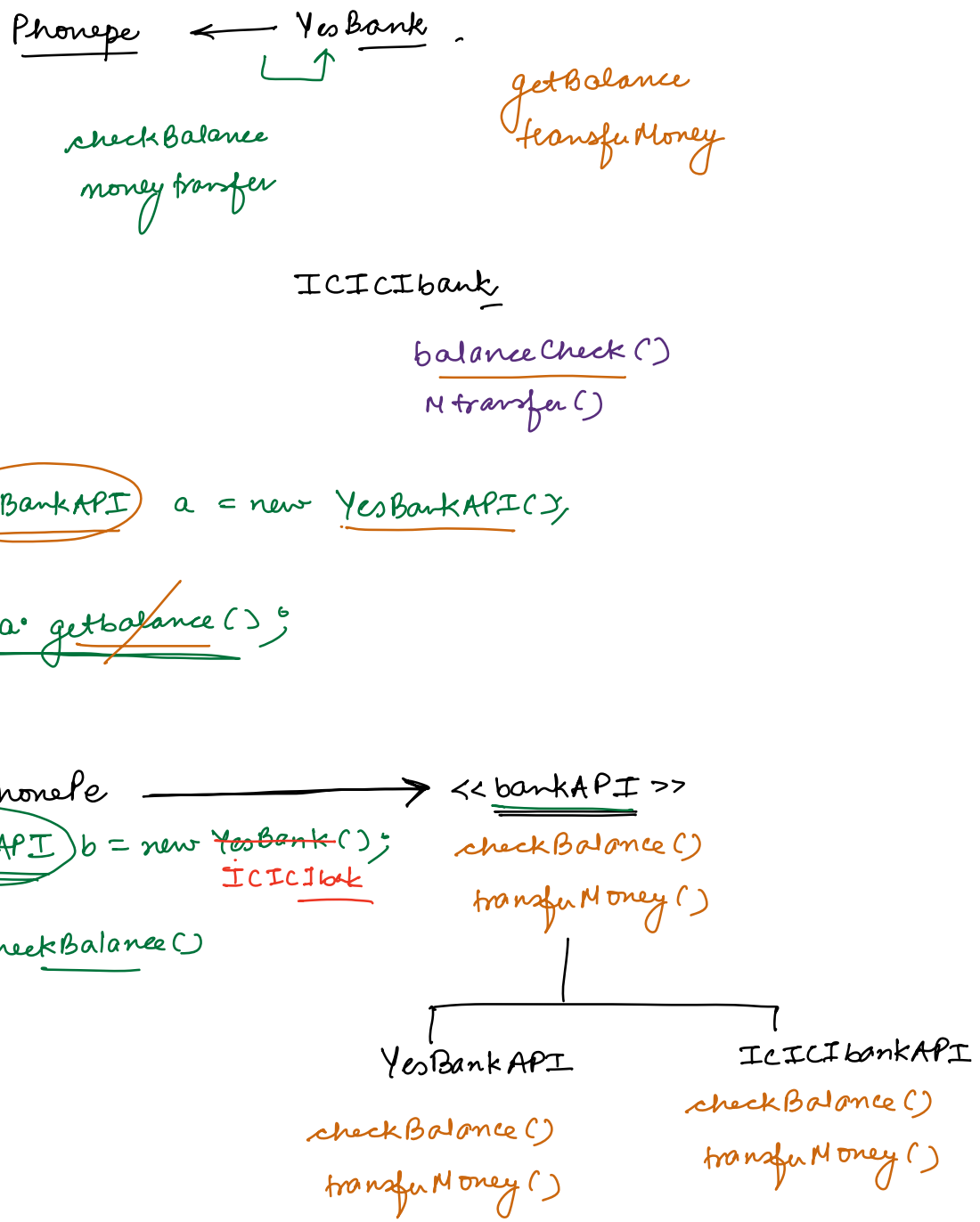


collection

```
Stack s = new ArrayStack()
        = new LinkedStack()
        = new QueueStack()
```

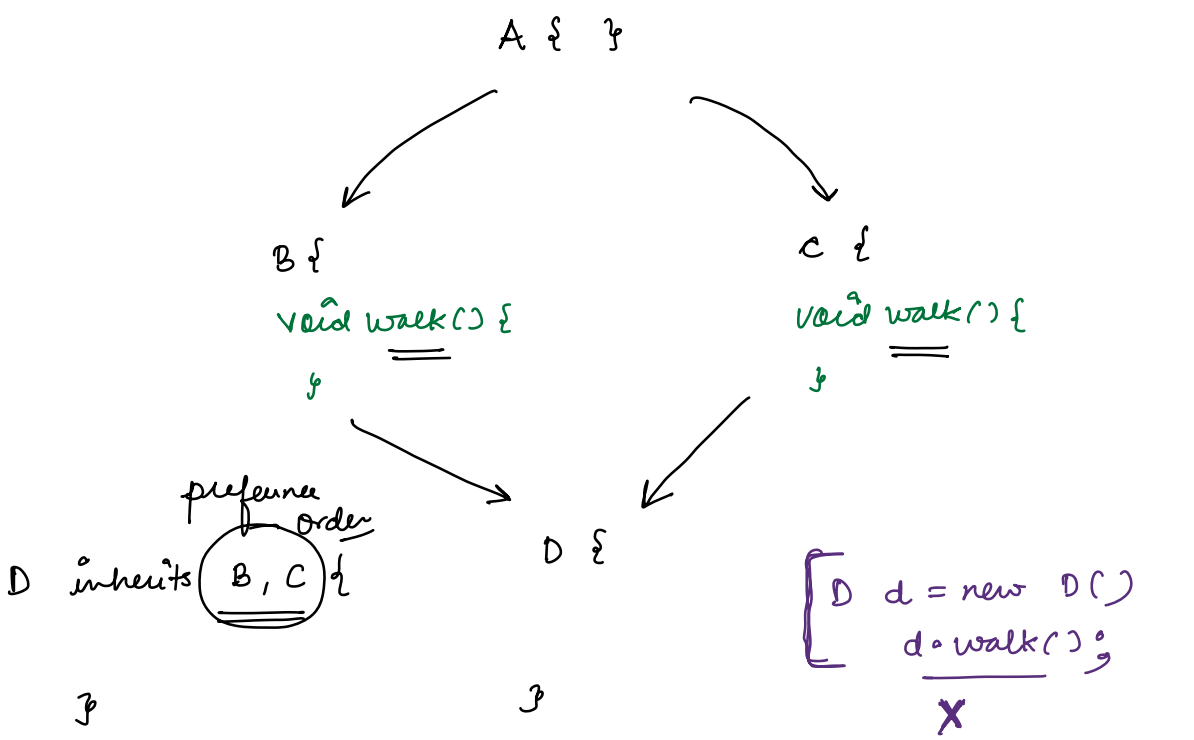
Abstraction is achieved here, because we are concerned about stack not actual implementation.

Abstraction
```
s.push()
s.pop()
```



"code to interfaces not implemetation".

classes can implement multiple interfaces.



default methods

interface abc {

default temp() {
= def";
}

xyz();

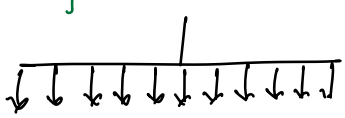
temp();

}

interface abcplus
extends abc {

temp();

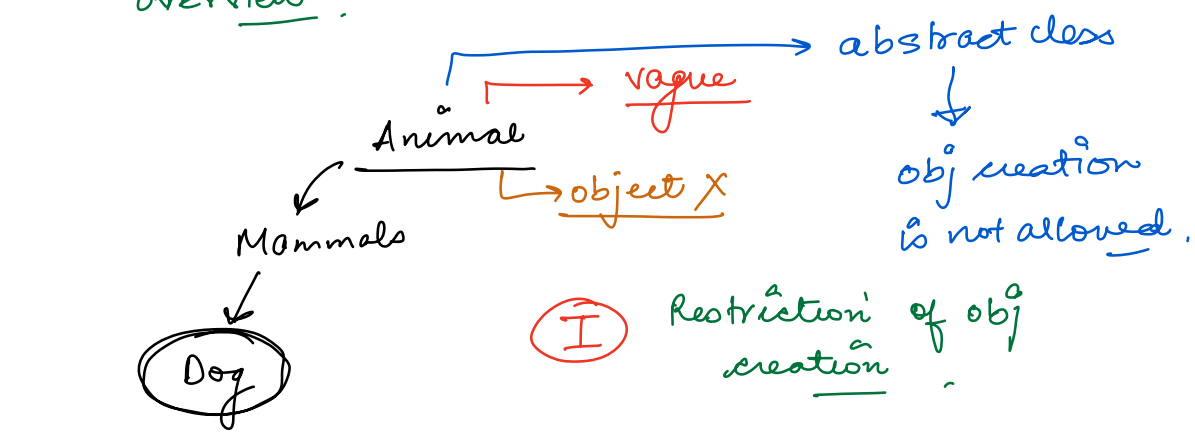
}



All the interfaces methods has to be public.

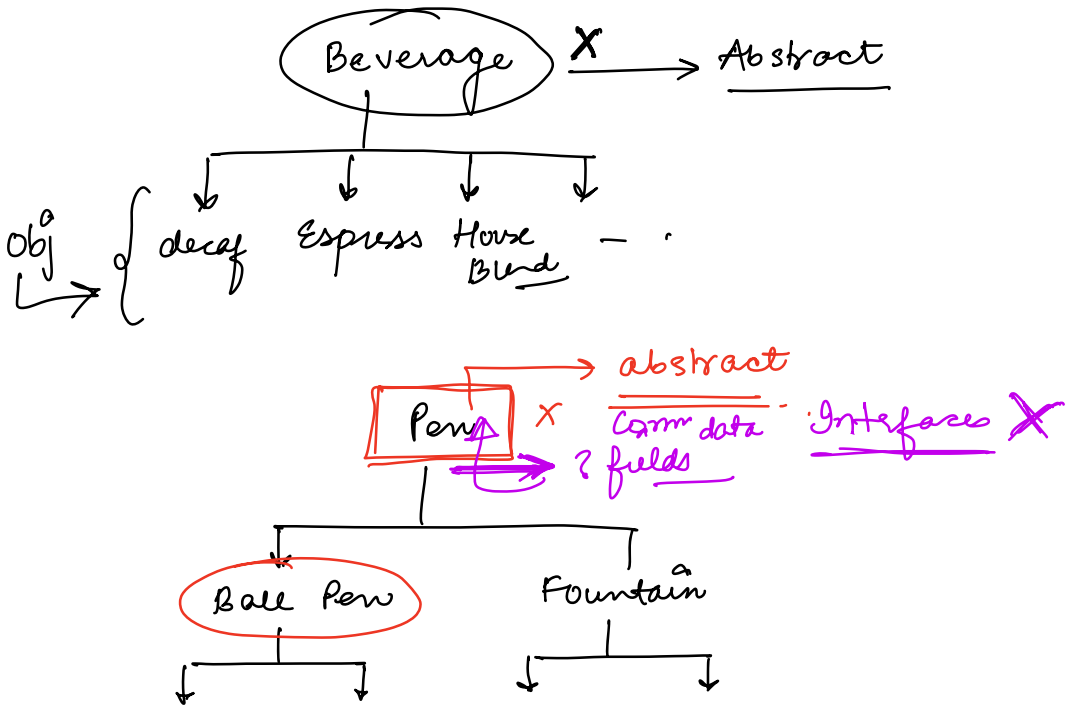
Abstract classes

high level
vague
overview.



int id;
Date createdAt;
Date ModifiedAt;

}



(II)

Abstract classes

- normal data members
- normal member fn
- abstract methods

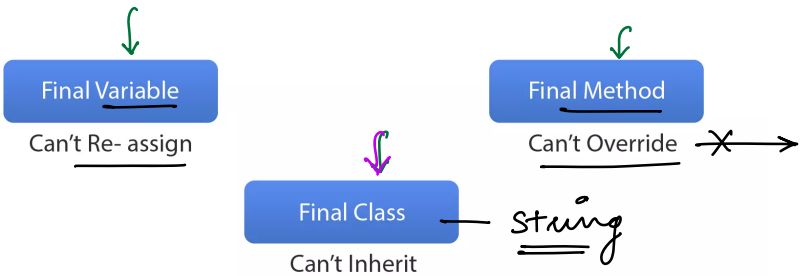
declaration — classes (abstract)

def^n will be in the class which will inherit

abstract void read();

✓

Final Keyword



SCALER Topics

final int x = 10; → constant